

Assignment -5

Name: Aditya Ingale

Roll no.: - 260250320005

1. Create two lists: Student and Course. Join them to display product names along with their category names using basic LINQ Queries.

```
namespace Assignment5
{
    internal class Student
    {
        public int Id;
        public string Name;
        public int CourseId; // links to Course.Id

        public Student(int id, string name, int courseId)
        {
            Id = id;
            Name = name;
            CourseId = courseId;
        }
    }
}

namespace Assignment5
{
    internal class Course
    {
        public int Id;
        public string CourseName;

        public Course(int id, string courseName)
        {
            Id = id;
            CourseName = courseName;
        }
    }
}

static void Main(string[] args)
{
    List<Student> students = new List<Student>
    {
        new Student(1, "Ravi Kumar", 101),
        new Student(2, "Priya Singh", 102),
        new Student(3, "Arjun Mehta", 101),
        new Student(4, "Sneha Patel", 103),
        new Student(5, "Kiran Rao", 102)
    };

    List<Course> courses = new List<Course>
    {
        new Course(101, "Computer Science"),
        new Course(102, "Electronics"),
        new Course(103, "Mathematics")
    };

    // LINQ Join
```

```

var studentCourses = from s in students
                      join c in courses on s.CourseId equals c.Id
                      select new { s.Name, c.CourseName };

Console.WriteLine("---- Students and their Courses ----");
foreach (var item in studentCourses)
{
    Console.WriteLine($"{item.Name} is enrolled in {item.CourseName}");
}
}

```

```

---- Students and their Courses ----
Ravi Kumar is enrolled in Computer Science
Priya Singh is enrolled in Electronics
Arjun Mehta is enrolled in Computer Science
Sneha Patel is enrolled in Mathematics
Kiran Rao is enrolled in Electronics

D:\Code\C#\Assignment5\bin\Debug\net8.0\Assignment5.exe (process 23264) exited with code 0 (0x0).
Press any key to close this window . . . |

```

2. Create a new console application, Define four classes: Customer, Product, Order, and OrderDetail. Add appropriate properties to each class.

Write LINQ queries to perform the following tasks:

- a. Display a list of all customers in alphabetical order by last name.
- b. Display a list of all products in order of unit price, from highest to lowest.
- c. Display a list of all orders that were shipped to customers in the Bangalore.

internal class Customer

```

{
    public int Id;
    public string FirstName;
    public string LastName;
    public string City;

    public Customer(int id, string firstName, string lastName, string city)
    {
        Id = id;
        FirstName = firstName;
        LastName = lastName;
        City = city;
    }
}

```

internal class Product

```

{
    public int Id;
    public string ProductName;
    public decimal UnitPrice;

    public Product(int id, string productName, decimal unitPrice)
    {
        Id = id;
        ProductName = productName;
        UnitPrice = unitPrice;
    }
}

```

internal class Order

```

    {
        public int Id;
        public int CustomerId;
        public string ShipCity;
        public DateTime OrderDate;

        public Order(int id, int customerId, string shipCity, DateTime orderDate)
        {
            Id = id;
            CustomerId = customerId;
            ShipCity = shipCity;
            OrderDate = orderDate;
        }
    }

    internal class OrderDetail
    {
        public int OrderId;
        public int ProductId;
        public int Quantity;

        public OrderDetail(int orderId, int productId, int quantity)
        {
            OrderId = orderId;
            ProductId = productId;
            Quantity = quantity;
        }
    }

    static void Main(string[] args)
    {
        List<Customer> customers = new List<Customer>
        {
            new Customer(1, "Ravi", "Kumar", "Bangalore"),
            new Customer(2, "Priya", "Singh", "Mumbai"),
            new Customer(3, "Arjun", "Mehta", "Bangalore"),
            new Customer(4, "Sneha", "Patel", "Delhi"),
            new Customer(5, "Kiran", "Rao", "Bangalore")
        };

        List<Product> products = new List<Product>
        {
            new Product(1, "Laptop", 65000.00m),
            new Product(2, "Mouse", 799.00m),
            new Product(3, "Keyboard", 1499.00m),
            new Product(4, "Monitor", 15000.00m),
            new Product(5, "Headphones", 2999.00m)
        };

        List<Order> orders = new List<Order>
        {
            new Order(1, 1, "Bangalore", new DateTime(2024, 1, 10)),
            new Order(2, 2, "Mumbai", new DateTime(2024, 2, 14)),
            new Order(3, 3, "Bangalore", new DateTime(2024, 3, 5)),
            new Order(4, 4, "Delhi", new DateTime(2024, 4, 20)),
            new Order(5, 5, "Bangalore", new DateTime(2024, 5, 18))
        };
    }

```

```
List<OrderDetail> orderDetails = new List<OrderDetail>
{
    new OrderDetail(1, 1, 1),
    new OrderDetail(1, 2, 2),
    new OrderDetail(2, 3, 1),
    new OrderDetail(3, 4, 2),
    new OrderDetail(4, 5, 3),
    new OrderDetail(5, 1, 1),
    new OrderDetail(5, 3, 2)
};
```

```
// — TASK 1: JOIN STUDENTS AND COURSES —————
```

```
Console.WriteLine("=====");
Console.WriteLine(" TASK 1 — Students and their Courses ");
Console.WriteLine("=====");
```

```
var studentCourses = from s in students
                      join c in courses on s.CourseId equals c.Id
                      select new { s.Name, c.CourseName };
```

```
foreach (var item in studentCourses)
{
    Console.WriteLine($" {item.Name,-15} → {item.CourseName}");
}
```

```
// — TASK 2a: CUSTOMERS ALPHABETICALLY BY LAST NAME —————
```

```
Console.WriteLine();
Console.WriteLine("=====");
Console.WriteLine(" TASK 2a — Customers (A→Z by Last Name) ");
Console.WriteLine("=====");
```

```
var sortedCustomers = customers.OrderBy(c => c.LastName);
```

```
foreach (var c in sortedCustomers)
{
    Console.WriteLine($" {c.LastName,-10}, {c.FirstName,-10} — {c.City}");
}
```

```
// — TASK 2b: PRODUCTS BY PRICE HIGH TO LOW —————
```

```
Console.WriteLine();
Console.WriteLine("=====");
Console.WriteLine(" TASK 2b — Products (Price: High → Low) ");
Console.WriteLine("=====");
```

```
var sortedProducts = products.OrderByDescending(p => p.UnitPrice);
```

```
foreach (var p in sortedProducts)
{
    Console.WriteLine($" {p.ProductName,-12} ₹ {p.UnitPrice,10:N2}");
}
```

```
// — TASK 2c: ORDERS SHIPPED TO BANGALORE —————
```

```

Console.WriteLine();
Console.WriteLine("=====");
Console.WriteLine(" TASK 2c — Orders Shipped to Bangalore ");
Console.WriteLine("=====");

```

```

var bangaloreOrders = from o in orders
    join c in customers on o.CustomerId equals c.Id
    where o.ShipCity == "Bangalore"
    select new
    {
        OrderId = o.Id,
        CustomerName = $"{c.FirstName} {c.LastName}",
        o.ShipCity,
        o.OrderDate
    };

```

```

foreach (var o in bangaloreOrders)
{
    Console.WriteLine($" Order #{o.OrderId} | {o.CustomerName,-15} | {o.ShipCity,-10} | {o.OrderDate:dd MMM
yyyy}");
}

```

// — BONUS: TOTAL QUANTITY ORDERED PER PRODUCT —————

```

Console.WriteLine();
Console.WriteLine("=====");
Console.WriteLine(" BONUS — Total Quantity per Product ");
Console.WriteLine("=====");

```

```

var quantityPerProduct = from d in orderDetails
    group d by d.ProductId into g
    join p in products on g.Key equals p.Id
    select new
    {
        p.ProductName,
        TotalQty = g.Sum(d => d.Quantity)
    };

```

```

foreach (var item in quantityPerProduct)
{
    Console.WriteLine($" {item.ProductName,-12} — Total ordered: {item.TotalQty}");
}

```

// — BONUS: PRODUCTS ORDERED BY BANGALORE CUSTOMERS —————

```

Console.WriteLine();
Console.WriteLine("=====");
Console.WriteLine(" BONUS — Products Ordered by Bangalore ");
Console.WriteLine("=====");

```

```

// Step 1: get order IDs that were shipped to Bangalore
var bangaloreOrderIds = orders
    .Where(o => o.ShipCity == "Bangalore")
    .Select(o => o.Id);

```

```
// Step 2: get order details for those orders, then join with products
```

```
var bangaloreProducts = from d in orderDetails
    where bangaloreOrderIds.Contains(d.OrderId)
    join p in products on d.ProductId equals p.Id
    select p.ProductName;
```

```
// Distinct() removes duplicates if same product appears in multiple orders
```

```
foreach (var name in bangaloreProducts.Distinct())
```

```
{
    Console.WriteLine($" {name}");
}
```

```
Console.WriteLine();
```

```
Console.WriteLine("=====");
```

```
Console.WriteLine(" All queries complete! ");
```

```
Console.WriteLine("=====");
```

```
}
```

TASK 2a - Customers (A-Z by Last Name)

Kumar	, Ravi	- Bangalore
Mehta	, Arjun	- Bangalore
Patel	, Sneha	- Delhi
Rao	, Kiran	- Bangalore
Singh	, Priya	- Mumbai

TASK 2b - Products (Price: High ? Low)

Laptop	? 65,000.00
Monitor	? 15,000.00
Headphones	? 2,999.00
Keyboard	? 1,499.00
Mouse	? 799.00

TASK 2c - Orders Shipped to Bangalore

Order #1	Ravi Kumar	Bangalore	10 Jan 2024
Order #3	Arjun Mehta	Bangalore	05 Mar 2024
Order #5	Kiran Rao	Bangalore	18 May 2024

BONUS - Total Quantity per Product

Laptop	- Total ordered: 2
Mouse	- Total ordered: 2
Keyboard	- Total ordered: 3
Monitor	- Total ordered: 2
Headphones	- Total ordered: 3

BONUS - Products Ordered by Bangalore

Laptop
Mouse
Monitor
Keyboard

All queries complete!

D:\Code\C#\Assignment5\bin\Debug\net8.0\Assignment5.exe (process 16256) exited with code 0 (0x0).
Press any key to close this window . . .|